REVISION CHEAT SHEET: THREADS / PARALLELISM

Threads, Parallelism, and Concurrency in C#

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. Concurrency vs Parallelism vs Asynchrony
   - Concurrency
   - Parallelism
   - Asynchrony

The Key Distinction

2. What Is a Thread?

Thread-Local vs Shared State

3. Creating a Raw Thread

Should You Normally Create Threads Yourself?

4. The Managed Thread Pool

Why Reuse Threads?

5. Task Is Not the Same as Thread

Mental Model

6. Task.Run

Don't Wrap Naturally Async I/O

ASP.NET Core Warning

7. Thread Safety

8. Race Condition

Race Condition Mental Model

9. Atomic Operations

Example

Useful Interlocked Operations

10. Interlocked Is Not a General Replacement for Locks

11. Critical Section

12. lock

Traditional Lock Object

13. Use a Dedicated Private Lock

14. Keep Critical Sections Small

15. You Cannot await Inside lock

16. Async Mutual Exclusion with SemaphoreSlim

Why Count = 1?

Semaphore with More Permits

17. lock vs SemaphoreSlim

18. Monitor

When Monitor Matters

19. volatile

Important: volatile Does Not Make Compound Operations Atomic

Visibility vs Atomicity

20. Reader/Writer Coordination

21. Mutex and Other Primitives

22. Deadlock

Deadlock Diagram

23. Preventing Deadlocks

24. Livelock

25. Starvation

26. Prefer Avoiding Shared Mutable State

Example

Easier

27. Thread Confinement

28. Parallel Programming

Parallelism Diagram

29. Parallel Does Not Mean Faster

Good Candidate

Poor Candidate

30. Parallel.For

31. Parallel.ForEach

32. Shared State in Parallel Loops

Better

33. Control Degree of Parallelism

Don't Automatically Use ProcessorCount Everywhere

34. Cancellation of Parallel Loops

35. Exceptions in Parallel Loops
36. Parallel.ForEachAsync
37. Bounded Async Concurrency
Why Not Create 50,000 Tasks?
38. Parallel.ForAsync
39. Task.WhenAll vs Parallel.ForEachAsync
- Task.WhenAll
- Parallel.ForEachAsync
40. Parallel LINQ — PLINQ
When PLINQ Fits
When PLINQ Does Not Fit
41. PLINQ Ordering
42. Do Not Parallelize EF Core on One DbContext
Correct Simple Version
43. DbContext Is Not Thread-Safe
44. Parallelizing After Materialization
Flow
45. ASP.NET Core Is Already Concurrent
46. CPU Oversubscription
47. Lock Contention
Think in Terms of Contention
48. Coarse-Grained vs Fine-Grained Locks
- Coarse Lock
- Fine-Grained Locks
49. Optimistic vs Pessimistic Coordination
50. In-Process Concurrency vs Database Concurrency
- Thread concurrency
- Database concurrency
51. Process-Wide Static Locks Don't Scale Across Servers
52. Shared Collections
53. Better: Return Independent Results
54. Thread-Local State
55. Do Not Assume Same Thread After await
56. Thread Affinity
57. Memory Visibility
58. “It Worked in Testing” Means Very Little for Races
59. Thread Safety Is Transitive Through Dependencies
60. Security Considerations
Authentication Context
61. Performance Considerations
Measure Before and After
62. Common Mistake - THREADS / PARALLELISM
Common Mistake — Locking this
Common Mistake — Huge Critical Section
Common Mistake — Await Inside Lock
Common Mistake — Forgetting Semaphore Release
Common Mistake — Over-Releasing SemaphoreSlim
Common Mistake — Using volatile for Counter Increment
Common Mistake — Parallelizing Tiny Work
Common Mistake — Shared List in Parallel.ForEach
Common Mistake — Assuming Ordering
Common Mistake — Same DbContext in Parallel
Common Mistake — Task.Run Around Database Query
Common Mistake — Static Mutable Request State
Common Mistake — Parallel Everything in ASP.NET Core
Common Mistake — Using Thread.Sleep for Coordination
Common Mistake — DIY Lock-Free Algorithm
Best Practices
Practical Exercise 1 — Fix the Counter
Practical Exercise 2 — Protect Multi-Step State
Practical Exercise 3 — Async Throttling
Practical Exercise 4 — CPU Parallelism
Practical Exercise 5 — Limit Parallel CPU Work
Practical Exercise 6 — Parallel Async Calls
Practical Exercise 7 — Fix a Deadlock Risk
Practical Exercise 8 — Keep EF Core Sequential on One Context
Practical Exercise 9 — Materialize Then Parallelize CPU Work
Practical Exercise 10 — Identify the Right Tool
Exactly 25 Interview Questions
Quick Revision
Complete Fundraising Example
Stage 1 — Database I/O
Stage 2 — CPU Calculation
Stage 3 — External HTTP Notifications
End-to-End Flow
Final Decision Flow
Final Concurrency Mental Model

Concurrency becomes important when more than one operation can make pr
period of time.

Examples include:

EXAMPLE / FLOW

```
Multiple ASP.NET Core requests
Multiple Tasks
Background workers
Parallel calculations
Multiple threads reading shared state
Several operations accessing the same cache
```

The difficulty is not merely making work happen simultaneously. The real challenge is preserving:

EXAMPLE / FLOW

```
Correctness
Consistency
Performance
Predictability
```

when operations overlap.

.NET generally recommends higher-level abstractions such as the **Task Parallel Library (TPL)** and PLINQ instead of manually managing raw threads for ordinary application work. Both ultimately use the managed thread pool for much of their execution.

1. Concurrency vs Parallelism vs Asynchrony

These three concepts are related but different.

Concurrency

Concurrency means:

> More than one operation is in progress during overlapping periods of time.

For example:

EXAMPLE / FLOW

```
Request A ──────────────────
        Request B ──────────────────
              Request C ──────────────
```

The operations overlap.

They don't necessarily execute simultaneously on different CPU cores.

Parallelism

Parallelism means:

> Two or more operations are actually executing at the same instant.

For example:

EXAMPLE / FLOW

```
CPU Core 1 → Calculate report A

CPU Core 2 → Calculate report B

CPU Core 3 → Calculate report C
```

This is mainly valuable for:

EXAMPLE / FLOW

```
CPU-bound work
```

Asynchrony

Asynchrony mainly concerns:

> ⓘ Waiting without blocking a thread.

Example:

C#

```
Gift? gift =
    await dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId == giftId,
            cancellationToken);
```

While SQL Server processes the query:

EXAMPLE / FLOW

```
No application thread needs
to sit blocked waiting.
```

The Key Distinction

EXAMPLE / FLOW

```
Async
    →
Efficient waiting

Concurrency
    →
Overlapping operations

Parallelism
    →
Simultaneous CPU execution
```

A `Task.WhenAll` over HTTP calls may provide concurrency without CPU parallelism.

A `Parallel.For` over expensive calculations is intended to exploit parallel CPU execution.

2. What Is a Thread?

A thread is an execution path through a process.

Within one .NET process:

EXAMPLE / FLOW

```
Process
 ├─ Thread 1
 ├─ Thread 2
 ├─ Thread 3
 └─ ...
```

Each thread has its own execution stack but threads within the same process can access the same managed heap.

That shared memory is what makes threading both useful and dangerous.

Thread-Local vs Shared State

Consider:

C#

```
void Calculate()
{
    int localCounter = 0;
}
```

Each invocation has its own local execution state.

But:

C#

```
private int _totalProcessed;
```

might be shared by several threads using the same object.

Then:

EXAMPLE / FLOW

```
Thread A ─┐
          ├─ _totalProcessed
Thread B ─┤
Thread C ─┘
```

Concurrent access now needs careful reasoning.

3. Creating a Raw Thread

You can explicitly create one:

C#

```
Thread thread =
    new(
        () =>
        {
            Console.WriteLine(
                $"Thread: {Environment.CurrentManagedThreadId}");
        });

thread.Start();

thread.Join();
```

`Start()` begins execution.

`Join()` blocks until that thread terminates.

Should You Normally Create Threads Yourself?

Usually not for ordinary .NET application work.

.NET recommends using:

EXAMPLE / FLOW

```
Task

Task.Run

Parallel.For

Parallel.ForEach

PLINQ
```

for most multithreaded applications because these abstractions allow the runtime to manage scheduling and reuse threads.

Explicit `Thread` can still be appropriate when you truly require characteristics such as:

EXAMPLE / FLOW

```
A dedicated long-lived thread

Specific thread priority

Apartment state

Specialized thread-affinity behavior
```

but it isn't the normal starting point for an ASP.NET Core application.

4. The Managed Thread Pool

Creating operating-system threads repeatedly is relatively expensive.

.NET therefore maintains a reusable:

EXAMPLE / FLOW

```
ThreadPool
```

Conceptually:

EXAMPLE / FLOW

```
Application work
      ↓
ThreadPool queue
      ↓
 ┌─────────┬─────────┬─────────┐
 ↓         ↓         ↓
Worker   Worker   Worker
Thread   Thread   Thread
```

The pool dynamically manages worker availability instead of applications creating a new thread for every unit of work. Tasks and many TPL operations use thread-pool threads by default.

Why Reuse Threads?

Without pooling:

EXAMPLE / FLOW

```
Create thread
Run tiny operation
Destroy thread
Create thread
Run next operation
Destroy thread
...
```

With pooling:

EXAMPLE / FLOW

```
Create managed workers
      ↓
Reuse workers
      ↓
Schedule many operations
```

This reduces thread-management overhead.

5. Task Is Not the Same as Thread

From Topic 27:

C#

```
Task task = Task.Run(CalculateReport);
```

doesn't mean:

EXAMPLE / FLOW

```
Task = Thread
```

A Task represents:

EXAMPLE / FLOW

```
An operation
```

whereas a Thread represents:

EXAMPLE / FLOW

```
An execution resource
```

A Task may execute on a thread-pool thread.

Different portions of asynchronous work can execute using different threads.

One thread may execute many Tasks over time.

Mental Model

EXAMPLE / FLOW

```
Task
    →
"What work must complete?"
```

EXAMPLE / FLOW

```
Thread
    →
"Where is code currently executing?"
```

THREADS / PARALLELISM

6. Task.Run

`Task.Run` queues work to the thread pool.

C#

```
decimal result = await Task.Run(() => CalculateGiftForecast());
```

This is most useful when the work is:

EXAMPLE / FLOW

```
CPU-bound
```

and the caller has a reason to move that computation away from its current execution context.

Don't Wrap Naturally Async I/O

Bad:

C#

```
await Task.Run(
    async () =>
        await httpClient.GetAsync(
            uri,
            cancellationToken));
```

`HttpClient.GetAsync` already supports true asynchronous I/O.

Use:

C#

```
await httpClient.GetAsync(
    uri,
    cancellationToken);
```

Wrapping it in `Task.Run` adds scheduling without making the I/O more asynchronous.

ASP.NET Core Warning

In server code:

C#

```
await Task.Run(
    () =>
        ExpensiveCpuOperation());
```

doesn't create more CPU capacity.

It moves work from one thread-pool thread to another.

ASP.NET Core recommends avoiding unnecessary thread-pool blocking and thread-pool offloading of request work because it can reduce scalability.

7. Thread Safety

A piece of code is thread-safe when it behaves correctly even when multiple threads access it concurrently according to its documented usage.

Consider:

```C#
public sealed class GiftCounter
{
    private int _count;

    public void Increment()
    {
        _count++;
    }

    public int Count =>
        _count;
}
```

This looks harmless.

It isn't thread-safe for concurrent increments.

8. Race Condition

Suppose:

```
EXAMPLE / FLOW
_count = 10
```

Two threads execute:

```C#
_count++;
```

The operation is conceptually:

```
EXAMPLE / FLOW
Read current value

Add 1

Write new value
```

Thread A:

```
EXAMPLE / FLOW
Read 10
```

Thread B:

```
EXAMPLE / FLOW
Read 10
```

Thread A:

EXAMPLE / FLOW

```
Write 11
```

Thread B:

EXAMPLE / FLOW

```
Write 11
```

Expected:

EXAMPLE / FLOW

```
12
```

Actual:

EXAMPLE / FLOW

```
11
```

One update is lost.

This is a race condition.

Race Condition Mental Model

EXAMPLE / FLOW

```
Thread A       Shared value       Thread B

Read 10  ←———— 10 ————→ Read 10

+1                          +1

Write 11 ————→ 11

                   11 ←——— Write 11
```

The final result depends on timing.

That is the core danger of shared mutable state.

9. Atomic Operations

An operation is atomic when other threads cannot observe it as partially completed.

This:

C#

```
_count++;
```

is a compound read-modify-write operation and should not be assumed atomic for synchronization purposes.

For simple shared counters:

C#

```
Interlocked.Increment(
    ref _count);
```

is appropriate.

`Interlocked` supplies atomic operations such as increment, add, exchange, and compare-exchange.

Example

C#

```
public sealed class GiftCounter
{
    private int _count;

    public void Increment()
    {
        Interlocked.Increment(
            ref _count);
    }

    public int Count =>
        Volatile.Read(
            ref _count);
}
```

Now the increment itself is atomic.

Useful Interlocked Operations

Typical operations include:

C#

```
Interlocked.Increment(
    ref value);

Interlocked.Decrement(
    ref value);

Interlocked.Add(
    ref value,
    amount);

Interlocked.Exchange(
    ref value,
    replacement);

Interlocked.CompareExchange(
    ref value,
    replacement,
    expected);
```

`CompareExchange` atomically compares a location with an expected value and conditionally replaces it, returning the original value.

10. Interlocked Is Not a General Replacement for Locks

Suppose a Gift requires:

EXAMPLE / FLOW

```
Read current allocated amount

Check remaining amount

Add allocation
```

```
Update total
```

Those operations form a multi-step invariant.

You cannot simply make each individual integer or decimal assignment atomic and assume the overall business operation is safe.

You may need:

EXAMPLE / FLOW

```
One synchronization boundary
around the complete invariant
```

instead.

11. Critical Section

A critical section is code that accesses shared state and must not be executed concurrently by multiple threads.

Example:

C#

```
if (
    _remainingAmount >= amount)
{
    _remainingAmount -= amount;

    _allocations.Add(
        allocation);
}
```

These steps belong together.

If two threads interleave them:

EXAMPLE / FLOW

```
Invariant may break.
```

We therefore protect the whole section.

12. lock

C# provides:

C#

```
lock
```

for mutual exclusion.

With current .NET:

C#

```
private readonly Lock _gate = new();

public void AddAllocation(
    GiftAllocation allocation)
{
    lock (_gate)
    {
```

```
        _allocations.Add(
            allocation);
    }
}
```

THREADS / PARALLELISM

Only one thread at a time can execute the protected region for that particular lock instance.

In .NET 9+/C# 13+, Microsoft recommends a dedicated `System.Threading.Lock` for ordinary C# synchronization. When the target is `System.Threading.Lock`, the compiler uses its specialized scope-based locking mechanism.

Traditional Lock Object

Older and still valid code often uses:

C#

```
private readonly object _gate = new();

lock (_gate)
{
    // Critical section.
}
```

For a normal reference-type lock target other than `System.Threading.Lock`, C# `lock` is conceptually implemented with:

EXAMPLE / FLOW

```
Monitor.Enter
    ↓
try
    ↓
critical section
    ↓
finally
    ↓
Monitor.Exit
```

ensuring the monitor is released even when an exception occurs.

13. Use a Dedicated Private Lock

Good:

C#

```
private readonly Lock _gate = new();
```

Avoid locking:

C#

```
lock (this)
```

because external code can also obtain the same object reference and lock it.

Also avoid using publicly accessible objects or interned strings as lock tokens.

A synchronization primitive should normally be:

EXAMPLE / FLOW

```
Private
```

```
Dedicated
*
Used only for synchronization
```

THREADS / PARALLELISM

14. Keep Critical Sections Small

Bad:

```C#
lock (_gate)
{
    UpdateState();

    CallRemoteApi();

    WriteFile();

    GenerateReport();
}
```

The lock remains held during:

EXAMPLE / FLOW

```
Network waiting

Disk waiting

Expensive calculation
```

Other threads cannot enter the protected region.

Better:

```C#
GiftSnapshot snapshot;

lock (_gate)
{
    UpdateState();

    snapshot = CreateSnapshot();
}

await SendSnapshotAsync(
    snapshot,
    cancellationToken);
```

Protect only the state mutation that actually needs synchronization.

15. You Cannot `await` Inside `lock`

This is invalid:

```C#
lock (_gate)
{
    await SaveAsync();
}
```

C# disallows `await` in a `lock` body.

This restriction exists for good reason: a normal monitor lock is tied to synchronous lock ownership semantics and should not be held while an async method suspends.

16. Async Mutual Exclusion with SemaphoreSlim

If asynchronous code genuinely needs to serialize access:

C#
```
private readonly SemaphoreSlim
    _gate = new(initialCount: 1, maxCount: 1);

public async Task ProcessAsync(
    CancellationToken cancellationToken)
{
    await _gate.WaitAsync(
        cancellationToken);

    try
    {
        await PerformAsyncWork(
            cancellationToken);
    }
    finally
    {
        _gate.Release();
    }
}
```

`WaitAsync` waits asynchronously until the semaphore can be entered, rather than blocking a thread.

Why Count = 1?

C#
```
new SemaphoreSlim(
    1,
    1);
```

means:

EXAMPLE / FLOW
```
At most one caller
inside at once.
```

That provides asynchronous mutual exclusion.

Semaphore with More Permits

C#
```
new SemaphoreSlim(
    5,
    5);
```

means:

EXAMPLE / FLOW
```
At most 5 concurrent callers.
```

This is useful for:

```
API throttling

Limited database-independent operations

External service protection

Batch processing
```

17. lock vs SemaphoreSlim

Requirement	Typical Choice
Short synchronous critical section	`lock`
Simple synchronous shared-state protection	`lock`
Atomic counter	`Interlocked`
Async code must wait for exclusive access	`SemaphoreSlim(1,1)`
Limit N concurrent async operations	`SemaphoreSlim(N,N)`

Do not use `SemaphoreSlim` everywhere simply because it supports async.

For ordinary synchronous memory protection:

C#
```
lock
```

is simpler and harder to misuse.

18. Monitor

`Monitor` is the lower-level synchronization primitive historically underlying ordinary object-based `lock`.

You can manually write:

C#
```
Monitor.Enter(
    _gate);

try
{
    // Critical section.
}
finally
{
    Monitor.Exit(
        _gate);
}
```

But normally prefer:

C#
```
lock (_gate)
{
}
```

because it is clearer and safely generates the required release pattern. Microsoft recommends the C# `lock` statement over directly manipulating `Monitor` for ordinary mutual exc

When Monitor Matters

`Monitor` exposes advanced operations including:

EXAMPLE / FLOW

```
TryEnter

Wait

Pulse

PulseAll
```

These can build condition-based coordination.

They are lower-level mechanisms and are rarely the first choice for modern application code.

19. volatile

The `volatile` keyword tells the compiler/runtime that accesses to a field require special memory-visibility semantics.

Example:

C#

```
private volatile bool _stopping;
```

One thread might write:

C#

```
_stopping = true;
```

another reads it.

Important: volatile Does Not Make Compound Operations Atomic

This is still unsafe:

C#

```
private volatile int _count;

_count++;
```

because:

EXAMPLE / FLOW

```
Read
+
Increment
+
Write
```

is still a multi-step operation.

Microsoft recommends using `Interlocked`, `lock`, or higher-level synchronization for most concurrency scenarios rather than relying on `volatile`.

Visibility vs Atomicity

Think:

EXAMPLE / FLOW

```
volatile
    →
Visibility / ordering semantics
for the field access
```

EXAMPLE / FLOW

```
Interlocked
    →
Atomic single-variable operations
```

EXAMPLE / FLOW

```
lock
    →
Atomicity for a larger critical section
+
memory synchronization
```

20. Reader/Writer Coordination

Sometimes an application has:

EXAMPLE / FLOW

```
Many readers

Few writers
```

.NET provides:

C#

```
ReaderWriterLockSlim
```

which can allow multiple readers concurrently while writers receive exclusive access.

Conceptually:

EXAMPLE / FLOW

```
Reader A ─┐
Reader B ─┼─ allowed together
Reader C ─┘

Writer
    →
must be exclusive
```

This can be useful for highly read-heavy shared structures.

But it adds complexity.

For most business application code:

EXAMPLE / FLOW

```
ordinary lock
```

THREADS / PARALLELISM

is a better starting point unless measurement demonstrates a need for reader/writer coordination.

21. Mutex and Other Primitives

.NET provides several other synchronization mechanisms:

EXAMPLE / FLOW

```
Mutex
Semaphore / SemaphoreSlim
ReaderWriterLockSlim
ManualResetEventSlim
AutoResetEvent
CountdownEvent
Barrier
SpinLock
SpinWait
```

They serve specialized coordination needs.

For typical ASP.NET Core application code you will most often encounter:

EXAMPLE / FLOW

```
lock

Interlocked

SemaphoreSlim
```

The others matter more in specialized threading infrastructure.

22. Deadlock

A deadlock occurs when operations wait for one another in a cycle and none can make progress.

Example:

C#

```
lock (_giftLock)
{
    lock (_campaignLock)
    {
        // Work.
    }
}
```

Elsewhere:

C#

```
lock (_campaignLock)
{
    lock (_giftLock)
    {
        // Work.
    }
}
```

Possible execution:

EXAMPLE / FLOW

```
Thread A
    owns GiftLock
    waits CampaignLock

Thread B
    owns CampaignLock
    waits GiftLock
```

Neither can continue.

Deadlock Diagram

EXAMPLE / FLOW

```
Thread A
    ↓ owns
GiftLock
    ↓ waits for
CampaignLock
    ↑ owns
Thread B
    ↑ waits for
GiftLock
```

This cycle never resolves automatically.

23. Preventing Deadlocks

A simple strategy is consistent lock ordering.

For example:

EXAMPLE / FLOW

```
Always acquire:

1. GiftLock
2. CampaignLock
```

Never reverse the order.

Also:

EXAMPLE / FLOW

```
Keep locks narrow

Avoid nested locks where possible

Don't call unknown external code while holding locks

Don't block on async operations while holding locks

Use timeouts where an appropriate primitive supports them
```

24. Livelock

A livelock differs from deadlock.

Deadlock:

EXAMPLE / FLOW

```
Everyone is blocked.
```

Livelock:

EXAMPLE / FLOW

```
Everyone is active,
but repeatedly reacts to each other
without making progress.
```

Imagine two processes continually backing away for one another and retrying at the same time.

Concurrency correctness means:

EXAMPLE / FLOW

```
No races

No deadlocks

No livelocks

Acceptable fairness/progress
```

25. Starvation

Starvation means one operation rarely or never gets access to the resource it needs because other operations continually win.

Example:

EXAMPLE / FLOW

```
Thread A keeps acquiring resource
Thread B keeps acquiring resource
Thread C keeps waiting
```

Thread C may remain alive but never progress.

Thread-pool starvation is another important server problem, commonly caused by excessive blocking. ASP.NET Core specifically warns that blocking thread-pool threads reduces application throughput.

26. Prefer Avoiding Shared Mutable State

The easiest lock to reason about is often:

EXAMPLE / FLOW

```
No lock at all.
```

Designs become substantially easier when data is:

EXAMPLE / FLOW

```
Immutable

Local

Request-scoped

Copied

Owned by one operation
```

instead of globally shared and mutated.

Example

More difficult:

```C#
private static List<Gift>
    _gifts = [];
```

Every request can touch the same mutable list.

Easier

```C#
public async Task ProcessAsync()
{
    List<GiftSnapshot> gifts = await LoadAsync();

    // This list belongs only to this operation.
}
```

Now no other request necessarily shares that local list.

27. Thread Confinement

Thread confinement means state is accessed only by one thread or one isolated execution owner.

Even more generally:

```EXAMPLE / FLOW
Operation confinement
```

can prevent shared-state problems.

Example:

```EXAMPLE / FLOW
Worker owns its batch
    ↓
Nobody else mutates batch
    ↓
No synchronization required
```

This is often more scalable than surrounding every object with locks.

28. Parallel Programming

Parallel programming aims to divide CPU-bound work across multiple processors.

Suppose 1,000 Gift snapshots need a computationally expensive calculation.

Sequential:

```C#
foreach (
    GiftSnapshot gift
    in gifts)
{
    CalculateScore(
```

```
        gift);
}
```


Parallel:

```C#
Parallel.ForEach(
    gifts,
    gift =>
    {
        CalculateScore(
            gift);
    });
```

The Task Parallel Library partitions work and schedules it across available execution resources.

Parallelism Diagram

Sequential:

```EXAMPLE / FLOW
Core 1:
A → B → C → D
```

Potential parallel execution:

```EXAMPLE / FLOW
Core 1: A ————
Core 2: B ————
Core 3: C ————
Core 4: D ————
```

If each unit contains enough CPU work, elapsed time may decrease.

29. Parallel Does Not Mean Faster

Parallelization adds overhead:

```EXAMPLE / FLOW
Partition work

Schedule tasks

Synchronize workers

Combine results

Manage shared state
```

For tiny operations:

```C#
Parallel.For(
    0,
    10,
    i =>
    {
        int x = i + 1;
    });
```

parallel execution may be slower than a normal loop.

Microsoft explicitly warns not to assume parallel loops are faster and recomm

workloads.

Good Candidate

EXAMPLE / FLOW

```
Large number of iterations

Each iteration computationally expensive

Iterations mostly independent

Enough CPU cores available
```

Poor Candidate

EXAMPLE / FLOW

```
Very small collection

Tiny operation per item

Heavy shared-state synchronization

Mostly network/database waiting

Strict sequential dependency
```

30. Parallel.For

Instead of:

C#

```
for (
    int i = 0;
    i < gifts.Length;
    i++)
{
    CalculateScore(
        gifts[i]);
}
```

you can write:

C#

```
Parallel.For(
    0,
    gifts.Length,
    i =>
    {
        CalculateScore(
            gifts[i]);
    });
```

Iterations may execute concurrently and potentially in parallel.

Do not assume iteration order.

TPL may partition the iteration range among participating tasks/threads.

31. Parallel.ForEach


```C#
Parallel.ForEach(
    gifts,
    gift =>
    {
        CalculateScore(
            gift);
    });
```

This is useful for independent per-element CPU work.

But:

```EXAMPLE / FLOW
Iteration 1
Iteration 2
Iteration 3
```

are not guaranteed to execute:

```EXAMPLE / FLOW
1 → 2 → 3
```

or even all simultaneously.

Microsoft specifically warns that correctness must not depend on parallel-loop execution order or every iteration actually running at the same time.

32. Shared State in Parallel Loops

Bad:

```C#
decimal total = 0m;

Parallel.ForEach(
    gifts,
    gift =>
    {
        total += gift.TotalAmount;
    });
```

Several threads may simultaneously modify:

```C#
total
```

creating a race.

Better

For supported numeric types you might use atomic operations, or better structure the algorithm around partition-local results and combine them afterward.

The general rule is:

Avoid shared writable variables inside parallel loops whenever

Microsoft explicitly recommends minimizing shared memory access in parallel loops.

33. Control Degree of Parallelism

You can limit parallelism:

C#

```
ParallelOptions options =
    new()
    {
        MaxDegreeOfParallelism =
            Environment.ProcessorCount,

        CancellationToken = cancellationToken };

Parallel.ForEach(
    gifts,
    options,
    gift =>
    {
        CalculateScore(
            gift);
    });
```

`MaxDegreeOfParallelism` is particularly useful when you need to avoid excessive resource contention.

Don't Automatically Use ProcessorCount Everywhere

For CPU work:

EXAMPLE / FLOW

```
ProcessorCount
```

is often a sensible starting point.

But ideal parallelism depends on:

EXAMPLE / FLOW

```
Work size

Memory bandwidth

Contention

Other application activity

Runtime scheduling

Environment
```

Measure.

34. Cancellation of Parallel Loops

C#

```
ParallelOptions options = new() { CancellationToken = cancellationToken };
```

```
Parallel.ForEach(
    gifts,
    options,
    gift =>
    {
        CalculateScore(
            gift);
    });
```

When cancellation is requested, the loop cooperatively stops according to TPL semantics.

You can combine:

EXAMPLE / FLOW

```
Cancellation

MaxDegreeOfParallelism
```

through the same `ParallelOptions`.

35. Exceptions in Parallel Loops

Several iterations can fail independently.

C#

```
try
{
    Parallel.ForEach(
        gifts,
        ProcessGift);
}
catch (
    AggregateException ex)
{
    foreach (
        Exception error
        in ex.InnerExceptions)
    {
        Console.WriteLine(
            error.Message);
    }
}
```

TPL parallel loops can aggregate exceptions arising from concurrently executing iterations.

Do not simply suppress those failures.

Define whether:

EXAMPLE / FLOW

```
One failure aborts processing

Failures are collected

Failed items are retried

Partial results are acceptable
```

according to business requirements.

36. Parallel.ForEachAsync

When each element requires an async operation, don't write:

```C#
Parallel.ForEach(
    gifts,
    async gift =>
    {
        await SendReceiptAsync(
            gift);
    });
```

That callback doesn't give `Parallel.ForEach` proper awaitable semantics.

Use:

```C#
await Parallel.ForEachAsync(
    gifts,
    cancellationToken,
    async (
        gift,
        token) =>
    {
        await SendReceiptAsync(
            gift,
            token);
    });
```

`Parallel.ForEachAsync` accepts asynchronous delegates and returns a Task representing the entire operation. The default overload executes at most `Environment.ProcessorCount` operations in parallel.

37. Bounded Async Concurrency

For an external service:

```C#
ParallelOptions options =
    new()
    {
        MaxDegreeOfParallelism =
            8,

        CancellationToken = cancellationToken };

await Parallel.ForEachAsync(
    gifts,
    options,
    async (
        gift,
        token) =>
    {
        await receiptClient.SendAsync(
            gift.GiftId,
            token);
    });
```

Now at most eight operations are intended to be active through this loop's configured degree.

This can protect:

EXAMPLE / FLOW

```
External API rate limits

Connection pools

Memory

Downstream systems
```

THREADS / PARALLELISM

Why Not Create 50,000 Tasks?

This:

C#

```
await Task.WhenAll(
    gifts.Select(
        gift =>
            SendReceiptAsync(
                gift)));
```

may start an enormous number of operations immediately.

Sometimes that is fine for a small input.

For an unbounded input:

EXAMPLE / FLOW

```
Bounded concurrency
```

is often safer.

38. Parallel.ForAsync

Modern .NET also provides:

C#

```
Parallel.ForAsync
```

for asynchronous index-based parallel loops.

Example:

C#

```
await Parallel.ForAsync(
    0,
    gifts.Count,
    cancellationToken,
    async (
        index,
        token) =>
    {
        await ProcessAsync(
            gifts[index],
            token);
    });
```

`Parallel.ForAsync` is available in modern .NET and, with the default configuration, limits concurrent operations to approximately processor-count semantics.

For ordinary collection processing, `Parallel.ForEachAsync` is often more natural.

39. Task.WhenAll vs Parallel.ForEachAsync

THREADS / PARALLELISM

These can solve overlapping but different problems.

Task.WhenAll

C#
```
Task[] tasks =
    gifts
        .Select(
            gift =>
                SendAsync(
                    gift))
        .ToArray();

await Task.WhenAll(
    tasks);
```

Useful when:

EXAMPLE / FLOW
```
You already have a known set of Tasks

You intentionally want them started

The amount of concurrency is acceptable
```

Parallel.ForEachAsync

Useful when:

EXAMPLE / FLOW
```
You want iteration semantics

You want bounded parallelism

You want ParallelOptions

You want cancellation integrated
```

Neither is universally better.

Choose based on:

EXAMPLE / FLOW
```
Workload size

Dependency structure

Resource limits

Desired concurrency
```

40. Parallel LINQ — PLINQ

LINQ-to-Objects normally processes sequences sequentially:

C#
```
var results =
    gifts
        .Where(
```

```
            gift =>
                ExpensivePredicate(
                    gift))
        .Select(
            gift =>
                ExpensiveTransform(
                    gift));
```

PLINQ begins with:

```
C#
.AsParallel()
```

Example:

```
C#
var results =
    gifts
        .AsParallel()
        .Where(
            gift =>
                ExpensivePredicate(
                    gift))
        .Select(
            gift =>
                ExpensiveTransform(
                    gift))
        .ToList();
```

PLINQ attempts to parallelize supported query work across processors.

When PLINQ Fits

Good candidates:

```
EXAMPLE / FLOW
Large in-memory data

CPU-expensive predicates/projections

Independent elements

Few shared side effects
```

When PLINQ Does Not Fit

Poor candidates:

```
EXAMPLE / FLOW
EF Core IQueryable

Database I/O

Tiny sequences

Very cheap selectors

Order-sensitive side-effect processing
```

`AsParallel()` is for in-memory parallel LINQ, not a way to make SQL Server queries magically parallel from application code.

41. PLINQ Ordering

Parallel processing naturally makes completion order nondeterministic.

If source ordering matters:

```C#
gifts
    .AsParallel()
    .AsOrdered()
    .Where(...)
```

can preserve ordering where supported.

But ordering adds coordination overhead.

Microsoft recommends avoiding unnecessary ordering with PLINQ and measuring whether parallelization actually improves performance.

42. Do Not Parallelize EF Core on One DbContext

This deserves special emphasis for the Fundraising Platform.

Bad:

```C#
Task<Gift?> giftTask = _dbContext.Gifts.SingleOrDefaultAsync(gift => gift.GiftId == giftId);

Task<Campaign?> campaignTask =
    _dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                == campaignId);

await Task.WhenAll(
    giftTask,
    campaignTask);
```

Both queries use:

```EXAMPLE / FLOW
the same DbContext instance
```

concurrently.

EF Core explicitly does **not** support parallel operations on the same `DbContext`. Await one operation before starting another, or deliberately create separate contexts for truly independent parallel operations.

Correct Simple Version

```C#
Gift? gift =
    await _dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId == giftId,
            cancellationToken);
```

```
Campaign? campaign =
    await _dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                == campaignId, cancellationToken);
```

Sequential database operations on one context are safe and often perfectly appropriate.

43. DbContext Is Not Thread-Safe

Do not store a scoped context in some shared singleton and use it from multiple threads.

This creates:

EXAMPLE / FLOW

```
Concurrent use

Lifetime violations

Possible state corruption

Unexpected EF errors
```

The normal ASP.NET Core scoped `DbContext` lifetime is designed around one unit of work/request, not application-wide concurrent access.

44. Parallelizing After Materialization

Suppose you first load immutable calculation inputs:

C#

```
List<GiftCalculationInput> inputs =
    await _dbContext.Gifts
        .AsNoTracking()
        .Select(
            gift =>
                new GiftCalculationInput(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);
```

Then the database operation is complete.

If you now have expensive CPU calculations that don't use the context:

C#

```
Parallel.ForEach(
    inputs,
    input =>
    {
        CalculateForecast(
            input);
    });
```

that may be a valid parallelization candidate.

Flow

EXAMPLE / FLOW

```
SQL Server
    ↓
await query
    ↓
Materialize independent snapshots
    ↓
Database work finished
    ↓
CPU-intensive calculation
    ↓
Parallel.ForEach
```

This separation is much easier to reason about.

45. ASP.NET Core Is Already Concurrent

Suppose 100 users call your API.

ASP.NET Core may already be handling many requests concurrently.

Therefore this thinking is dangerous:

> ⓘ “I should parallelize every request to make the server faster.”

One request using eight CPU workers multiplied by hundreds of simultaneous requests can cause:

EXAMPLE / FLOW

```
CPU oversubscription

Thread-pool contention

Cache pressure

Memory pressure

Worse latency
```

Server-level throughput matters more than making one request complete slightly faster.

ASP.NET Core guidance recommends keeping hot paths asynchronous for I/O, avoiding blocking, and being careful with expensive CPU-bound work in request processing.

46. CPU Oversubscription

Suppose:

EXAMPLE / FLOW

```
8 CPU cores
```

and 100 requests each launch:

EXAMPLE / FLOW

```
8 parallel CPU tasks
```

You may now have:

EXAMPLE / FLOW

```
800 units of CPU-bound work
```

competing for eight cores.

More threads do not create more processors.

They may instead increase:

EXAMPLE / FLOW

```
Context switching

Scheduler overhead

Cache misses

Latency
```

47. Lock Contention

A lock prevents races but introduces serialization.

Suppose:

C#

```
lock (_gate)
{
    CalculateHugeReport();
}
```

Even with 32 threads:

EXAMPLE / FLOW

```
Only one thread can do work
inside this lock.
```

Your program has effectively created a bottleneck.

Think in Terms of Contention

Low contention:

EXAMPLE / FLOW

```
Thread enters
does tiny update
leaves
```

High contention:

EXAMPLE / FLOW

```
Many threads wait
for one long critical section
```

Synchronization is necessary for correctness, but excessive synchronization destroys parallel benefit.

48. Coarse-Grained vs Fine-Grained Locks

Coarse Lock

EXAMPLE / FLOW

```
One lock protects many things.
```

Advantages:

EXAMPLE / FLOW

```
Simpler correctness reasoning
```

Disadvantages:

EXAMPLE / FLOW

```
More contention
Less concurrency
```

Fine-Grained Locks

EXAMPLE / FLOW

```
Different locks protect
different pieces of state.
```

Advantages:

EXAMPLE / FLOW

```
More potential concurrency
```

Disadvantages:

EXAMPLE / FLOW

```
More complexity
More opportunities for deadlock
Harder invariants
```

Start simple.

Only introduce finer locking when measurements justify the complexity.

49. Optimistic vs Pessimistic Coordination

An in-process `lock` is essentially:

EXAMPLE / FLOW

```
Pessimistic coordination
```

because callers wait before touching protected state.

Another approach is optimistic:

EXAMPLE / FLOW

```
Read state

Compute proposed result

Atomically update only if state
hasn't changed
```

```
Retry if necessary
```


`Interlocked.CompareExchange` is a low-level building block for some lock-free algorithms.

Do not implement complex lock-free structures casually; they are much harder to prove correct.

50. In-Process Concurrency vs Database Concurrency

These are different concepts.

Thread concurrency

EXAMPLE / FLOW

```
Multiple application threads
touch the same memory
```

Solutions include:

EXAMPLE / FLOW

```
lock
Interlocked
SemaphoreSlim
confinement
```

Database concurrency

EXAMPLE / FLOW

```
Two requests/users update
the same database row
```

A C# lock in one process does not reliably solve this in a scaled-out system.

Example:

EXAMPLE / FLOW

```
Server A ─┐
          ├─ SQL Server
Server B ─┘
```

`lock` on Server A knows nothing about Server B.

Database-level techniques include:

EXAMPLE / FLOW

```
Transactions

Isolation

Concurrency tokens

Optimistic concurrency

Constraints
```

Those are a separate persistence concern.

51. Process-Wide Static Locks Don't Scale Across Servers

This:

```C#
private static readonly Lock
    _giftLock = new();
```

coordinates threads only inside that running process.

If your application has:

```EXAMPLE / FLOW
Instance A
Instance B
Instance C
```

each process has its own lock.

Therefore:

```EXAMPLE / FLOW
Static lock
```

is not a distributed lock.

Never use it as your only protection for a cross-server business invariant.

52. Shared Collections

From Topic 21:

```C#
List<Gift>
Dictionary<long, Gift>
HashSet<long>
```

aren't general-purpose containers for arbitrary concurrent mutation.

This is unsafe:

```C#
Parallel.ForEach(
    gifts,
    gift =>
    {
        results.Add(
            Process(
                gift));
    });
```

if:

```C#
results
```

is a normal `List<T>` shared by the iterations.

Topic 29 will introduce:

```EXAMPLE / FLOW
Concurrent collections

Immutable collections
```

for designs that genuinely need shared collection access.

53. Better: Return Independent Results

Rather than mutating one global list, consider transformations that produce independent results and combine them safely.

For CPU-bound LINQ:

```C#
GiftScore[] scores = gifts.AsParallel().Select(CalculateScore).ToArray();
```

Or maintain partition-local state with appropriate TPL overloads.

The broader principle is:

> **Prefer local state over synchronized shared state.**

54. Thread-Local State

Sometimes each worker can maintain its own local accumulator.

Conceptually:

```EXAMPLE / FLOW
Worker 1 → Local total 1
Worker 2 → Local total 2
Worker 3 → Local total 3
             ↓
        Combine at end
```

This is generally better than every iteration fighting over:

```EXAMPLE / FLOW
One global total
```

Microsoft's parallel-loop guidance recommends local state to reduce shared-memory contention.

55. Do Not Assume Same Thread After `await`

From Topic 27:

```C#
Console.WriteLine(
    Environment.CurrentManagedThreadId);

await SomeOperationAsync();

Console.WriteLine(
    Environment.CurrentManagedThreadId);
```

Those IDs may differ.

Async programming is about logical continuation, not preserving thread identity.

Therefore avoid designs that depend on:

EXAMPLE / FLOW

```
"This async method must always
continue on Thread 7."
```

unless you're deliberately working in a thread-affine environment.

56. Thread Affinity

Some technologies require work to happen on a specific thread.

Examples can include:

EXAMPLE / FLOW

```
WPF UI

Windows Forms UI

Certain COM components
```

In those environments:

EXAMPLE / FLOW

```
Background computation
    ↓
marshal update
    ↓
UI thread
```

is required.

TPL documentation specifically warns about thread-affinity requirements when parallelizing work.

ASP.NET Core does not have a traditional UI thread.

57. Memory Visibility

Processors and compilers can optimize memory access.

In multithreaded code, synchronization is not only about:

EXAMPLE / FLOW

```
"Who enters first?"
```

It also establishes important memory-ordering and visibility relationships.

Correct use of:

EXAMPLE / FLOW

```
lock

Interlocked

Volatile

Task synchronization primitives
```

provides the guarantees required by their documented semantics.

Do not try to invent your own memory synchronization using arbitrary fields and timing assumptions.

58. “It Worked in Testing” Means Very Little for Races

THREADS / PARALLELISM

A race may depend on:

EXAMPLE / FLOW

```
CPU speed

Processor count

Runtime optimization

Garbage collection timing

Request load

Operating system scheduling
```

This code might run successfully:

EXAMPLE / FLOW

```
1,000 times
```

then fail under production traffic.

Concurrency bugs are often:

EXAMPLE / FLOW

```
Intermittent

Timing-dependent

Hard to reproduce
```

Correctness must come from synchronization/design, not successful test timing.

59. Thread Safety Is Transitive Through Dependencies

Suppose:

C#

```
public sealed class GiftProcessor
{
    private readonly List<Gift>
        _gifts = [];
}
```

Even if every field assignment in `GiftProcessor` is atomic, concurrent calls to:

C#

```
_gifts.Add(...)
```

may still be unsafe.

Thread safety depends on:

EXAMPLE / FLOW

```
The whole operation

The collaborators it calls
```

```
The objects being shared
```

THREADS / PARALLELISM

not merely the containing class.

60. Security Considerations

Concurrency errors can become security problems.

Imagine a singleton mutable field:

```
C#
private long _currentTenantId;
```

Request A:

```
EXAMPLE / FLOW
Tenant 10
```

Request B:

```
EXAMPLE / FLOW
Tenant 20
```

If both requests modify the same shared field, one request might observe another tenant's value.

That could lead to:

```
EXAMPLE / FLOW
Cross-tenant data exposure
```

Never put request-specific security state in shared mutable singleton/static fields.

Authentication Context

Keep per-request information:

```
EXAMPLE / FLOW
Request-scoped

Method-local

Explicitly passed
```

rather than globally mutable.

Thread-local storage also isn't a substitute for properly scoped async request context because async code can resume on different threads.

61. Performance Considerations

Concurrency can improve:

```
EXAMPLE / FLOW
Throughput

Responsiveness

CPU utilization
```

but can also create:

EXAMPLE / FLOW

```
Lock contention

Context switching

Memory pressure

Cache contention

Scheduler overhead

Thread-pool starvation
```

Therefore:

> **Concurrency is not automatically a performance optimization.**

Measure Before and After

For parallelization:

EXAMPLE / FLOW

```
Sequential baseline
      ↓
Measure
      ↓
Parallel implementation
      ↓
Measure again
```

Microsoft specifically recommends measuring TPL/PLINQ performance rather than assuming parallel execution is faster.

62. Common Mistake — `counter++` Is Thread-Safe

It isn't.

Bad:

C#

```
_count++;
```

for a shared counter.

Better:

C#

```
Interlocked.Increment(
    ref _count);
```

Common Mistake — Locking `this`

Bad:

```C#
lock (this)
{
}
```

External code might lock the same instance.

Prefer:

```C#
private readonly Lock _gate = new();
```

Common Mistake — Huge Critical Section

Bad:

```C#
lock (_gate)
{
    UpdateState();

    SendHttpRequest();

    SaveFile();

    GeneratePdf();
}
```

Keep the synchronized region only as large as correctness requires.

Common Mistake — Await Inside Lock

Invalid:

```C#
lock (_gate)
{
    await WorkAsync();
}
```

Use an async-aware coordination primitive if asynchronous serialization is really necessary.

Common Mistake — Forgetting Semaphore Release

Bad:

```C#
await _semaphore.WaitAsync();

await ProcessAsync();

_semaphore.Release();
```

If `ProcessAsync()` throws:

EXAMPLE / FLOW

```
Release never occurs.
```

Better:

```C#
await _semaphore.WaitAsync();

try
{
    await ProcessAsync();
}
finally
{
    _semaphore.Release();
}
```

Common Mistake — Over-Releasing SemaphoreSlim

Every successful wait should correspond to the correct release.

Calling `Release` without owning a logical permit can corrupt your concurrency policy or exceed the configured maximum.

Common Mistake — Using volatile for Counter Increment

```C#
private volatile int _count;

_count++;
```

still has a race.

Use `Interlocked` or a lock.

Common Mistake — Parallelizing Tiny Work

```C#
Parallel.ForEach(
    numbers,
    number =>
        number++);
```

may be slower than:

```C#
foreach (...)
```

because scheduling/partitioning overhead exceeds computation.

Common Mistake — Shared List in Parallel.ForEach

Bad:

```C#
List<GiftScore> results = [];

Parallel.ForEach(
    gifts,
    gift =>
    {
        results.Add(
```

```
            CalculateScore(
                gift));
    });
```

THREADS / PARALLELISM

Use an appropriate result strategy rather than concurrently mutating an ordinary list.

Topic 29 will cover concurrent collection options.

Common Mistake — Assuming Ordering

Bad:

C#

```
Parallel.For(
    0,
    10,
    i =>
    {
        Console.WriteLine(
            i);
    });
```

and expecting:

EXAMPLE / FLOW

```
0
1
2
3
...
```

Parallel loops provide no such general ordering guarantee.

Common Mistake — Same DbContext in Parallel

Bad:

C#

```
await Task.WhenAll(
    QueryGiftAsync(
        _dbContext),
    QueryCampaignAsync(
        _dbContext));
```

EF Core doesn't permit parallel operations on the same context.

Common Mistake — Task.Run Around Database Query

Bad:

C#

```
await Task.Run(
    async () =>
        await dbContext.Gifts
            .ToListAsync());
```

The database API is already asynchronous.

Common Mistake — Static Mutable Request State

Bad:

```C#
public static long CurrentTenantId;
```

Multiple requests can overwrite one another.

Request-specific state shouldn't live in a shared static field.

Common Mistake — Parallel Everything in ASP.NET Core

A Web API is already processing many requests concurrently.

Per-request over-parallelization can reduce overall service throughput.

Common Mistake — Using Thread.Sleep for Coordination

Bad:

```C#
Thread.Sleep(
    100);
```

and hoping another thread has completed.

This is timing-based coordination.

Use proper:

```EXAMPLE / FLOW
Task

await

locks

semaphores

events/signals

cancellation
```

depending on the requirement.

Common Mistake — DIY Lock-Free Algorithm

Complex uses of:

```C#
CompareExchange
```

can be correct, but lock-free algorithms require careful reasoning about:

```EXAMPLE / FLOW
Atomicity

Memory ordering
```

Prefer proven framework primitives unless you have a measured reason to implement lower-level algorithms.

Best Practices

For ordinary shared synchronous state, start with:

```C#
private readonly Lock _gate = new();
```

and a small:

```C#
lock (_gate)
{
}
```

critical section.

For a simple shared counter, prefer:

```C#
Interlocked.Increment(...)
```

instead of locking an entire object.

For async throttling, use:

```C#
SemaphoreSlim.WaitAsync(...)
```

with `Release()` in `finally`.

Prefer avoiding shared mutable state altogether where practical.

Use immutable snapshots and method-local values.

Use `Task.Run` for appropriate CPU-bound offloading, not to wrap naturally async I/O.

Use `Parallel.For` / `Parallel.ForEach` only for suitable independent CPU-bound work.

Measure before assuming parallel execution improves performance.

Control degree of parallelism where resource limits matter.

Use `Parallel.ForEachAsync` for bounded asynchronous per-item operations when it fits the workload.

Do not access the same EF Core `DbContext` concurrently.

Do not assume execution order in parallel loops.

Keep lock ordering consistent when multiple locks are unavoidable.

Do not perform long I/O operations while holding synchronous locks.

Do not use `volatile` as a substitute for proper synchronization.

Keep security and tenant state request-scoped rather than globally mutable.

Remember that process-local synchronization does not enforce cross-server business invariants.

Practical Exercise 1 — Fix the Counter

Unsafe:

```C#
public sealed class Metrics
{
    private int _giftCount;

    public void Increment()
    {
        _giftCount++;
    }
}
```

Better:

```C#
public sealed class Metrics
{
    private int _giftCount;

    public void Increment()
    {
        Interlocked.Increment(
            ref _giftCount);
    }

    public int GiftCount =>
        Volatile.Read(
            ref _giftCount);
}
```

Practical Exercise 2 — Protect Multi-Step State

```C#
public sealed class GiftBatch
{
    private readonly Lock _gate = new();

    private readonly List<long>
        _giftIds = [];

    private decimal _total;

    public void Add(
        long giftId,
        decimal amount)
    {
        lock (_gate)
        {
            _giftIds.Add(
                giftId);

            _total += amount;
        }
    }
}
```

The list and total are updated as one protected operation.

Practical Exercise 3 — Async Throttling

C#

```
SemaphoreSlim throttle = new(5, 5);

async Task SendAsync(
    Gift gift,
    CancellationToken cancellationToken)
{
    await throttle.WaitAsync(
        cancellationToken);

    try
    {
        await receiptClient.SendAsync(
            gift.GiftId,
            cancellationToken);
    }
    finally
    {
        throttle.Release();
    }
}
```

No more than five callers can enter concurrently.

Practical Exercise 4 — CPU Parallelism

C#

```
Parallel.ForEach(
    giftSnapshots,
    snapshot =>
    {
        CalculateComplexForecast(
            snapshot);
    });
```

Use only when:

EXAMPLE / FLOW

```
Calculation is CPU-expensive

Snapshots are independent

Shared mutation is avoided
```

Practical Exercise 5 — Limit Parallel CPU Work

C#

```
ParallelOptions options =
    new()
    {
        MaxDegreeOfParallelism =
            Environment.ProcessorCount,

        CancellationToken = cancellationToken };

Parallel.ForEach(
    giftSnapshots,
```

```
    options,
    CalculateComplexForecast);
```


Practical Exercise 6 — Parallel Async Calls

```C#
ParallelOptions options =
    new()
    {
        MaxDegreeOfParallelism =
            8,

        CancellationToken = cancellationToken };

await Parallel.ForEachAsync(
    giftSnapshots,
    options,
    async (
        gift,
        token) =>
    {
        await receiptClient.SendAsync(
            gift.GiftId,
            token);
    });
```

Practical Exercise 7 — Fix a Deadlock Risk

Risk:

```C#
void First()
{
    lock (_giftLock)
    {
        lock (_campaignLock)
        {
        }
    }
}

void Second()
{
    lock (_campaignLock)
    {
        lock (_giftLock)
        {
        }
    }
}
```

Fix by using the same lock order everywhere:

```EXAMPLE / FLOW
Gift lock
    ↓
Campaign lock
```

or redesign to avoid needing both locks simultaneously.

Practical Exercise 8 — Keep EF Core Sequential on One Context


```C#
Gift? gift =
    await dbContext.Gifts
        .SingleOrDefaultAsync(
            gift =>
                gift.GiftId == giftId,
            cancellationToken);

Campaign? campaign =
    await dbContext.Campaigns
        .SingleOrDefaultAsync(
            campaign =>
                campaign.CampaignId
                == campaignId, cancellationToken);
```

Don't start both operations simultaneously on the same context.

Practical Exercise 9 — Materialize Then Parallelize CPU Work

```C#
List<GiftCalculationInput> inputs =
    await dbContext.Gifts
        .AsNoTracking()
        .Select(
            gift =>
                new GiftCalculationInput(
                    gift.GiftId,
                    gift.TotalAmount))
        .ToListAsync(
            cancellationToken);

Parallel.ForEach(
    inputs,
    input =>
    {
        CalculateScore(
            input);
    });
```

Database I/O and CPU parallelism have clear boundaries.

Practical Exercise 10 — Identify the Right Tool

Requirement:

```EXAMPLE / FLOW
Increment one shared integer
```

Use:

```EXAMPLE / FLOW
Interlocked
```

Requirement:

EXAMPLE / FLOW

```
Protect several related fields
synchronously
```

THREADS / PARALLELISM

Use:

EXAMPLE / FLOW

```
lock
```

Requirement:

EXAMPLE / FLOW

```
At most five async HTTP requests
at once
```

Use:

EXAMPLE / FLOW

```
SemaphoreSlim
or bounded Parallel.ForEachAsync
```

Requirement:

EXAMPLE / FLOW

```
CPU-heavy independent work
over many in-memory objects
```

Consider:

EXAMPLE / FLOW

```
Parallel.ForEach / PLINQ
```

Exactly 25 Interview Questions

1. What is the difference between concurrency and parallelism?

Concurrency means multiple operations make progress during overlapping periods.

Parallelism means multiple operations are literally executing simultaneously, usually on different CPU cores.

An application can be concurrent without being parallel.

2. What is the difference between asynchronous programming and parallel programming?

Asynchronous programming primarily avoids blocking while waiting for I/O.

Parallel programming primarily distributes CPU work across multiple processors.

For example:

C#

```
await httpClient.GetAsync(...)
```

is asynchronous I/O, while:

```C#
Parallel.ForEach(...)
```

is primarily a CPU parallelism construct.

3. What is a thread?

A thread is an execution path within a process.

Threads in the same process have their own execution stacks but can access shared process memory, which makes synchronization necessary when they mutate shared state.

4. What is the .NET ThreadPool?

The managed thread pool is a runtime-managed pool of reusable worker threads.

TPL operations and ordinary Task scheduling commonly use thread-pool threads, avoiding the expense and complexity of creating a dedicated thread for every operation.

5. Is a Task the same as a Thread?

No.

A Task represents an operation and its eventual completion.

A Thread is an execution resource.

A Task may execute on a thread-pool thread, but Tasks and Threads are not one-to-one concepts.

6. What is a race condition?

A race condition occurs when program correctness depends on the unpredictable timing/interleaving of concurrent operations.

For example:

```C#
_count++;
```

can lose updates if multiple threads perform the read-modify-write sequence concurrently.

7. What does atomic mean?

An atomic operation behaves as one indivisible operation with respect to competing threads.

`Interlocked` provides atomic operations such as increment, exchange, add, and compare-exchange for supported values.

8. When should you use `Interlocked`?

Use it for simple atomic operations on shared scalar state, such as counters or reference replacement.

Example:

```C#
Interlocked.Increment(
    ref _count);
```

It is not a replacement for a lock when several pieces of state must satisfy one invariant.

9. What does the `lock` statement do?

`lock` provides mutual exclusion around a synchronous critical section so only one thread holding the same lock can execute that region at a time.

With an ordinary reference target it uses monitor-style enter/exit semantics; with `System.Threading.Lock`, modern C# uses the specialized lock scope.

10. What should you lock on in modern C#?

For .NET 9+/C# 13+, a dedicated private:

```C#
System.Threading.Lock
```

is Microsoft's preferred ordinary C# locking primitive.

Avoid locking publicly accessible objects such as `this`.

11. Can you use `await` inside `lock`?

No.

C# prohibits `await` in the body of a `lock`.

If async code needs serialized access, an async-aware mechanism such as `SemaphoreSlim.WaitAsync` is commonly used instead.

12. What is `SemaphoreSlim` used for?

`SemaphoreSlim` limits how many callers may enter a protected region concurrently.

With count `1`, it can provide async-compatible mutual exclusion.

With count greater than one, it can throttle concurrent operations.

Its `WaitAsync` method allows asynchronous waiting.

13. What is the difference between `lock` and `SemaphoreSlim`?

`lock` is the straightforward choice for short synchronous critical sections.

`SemaphoreSlim` is useful when callers must wait asynchronously or when a limited number greater than one should enter concurrently.

14. What is a deadlock?

A deadlock occurs when operations wait on one another in a dependency cycle and none can proceed.

A common example is two threads acquiring the same two locks in opposite order.

Consistent lock ordering is one important prevention technique.

15. What are starvation and livelock?

Starvation means an operation repeatedly fails to obtain the resources required to progress.

Livelock means operations remain active and continually respond to one another but still make no useful progress.

Both differ from deadlock, where operations are blocked.

16. What does `volatile` do, and does it make code thread-safe?

`volatile` provides special visibility/ordering semantics for accesses to supported fields.

It does not make compound operations such as:

```C#
_count++;
```

atomic and doesn't eliminate general race conditions.

Microsoft recommends `Interlocked`, `lock`, or higher-level primitives for most synchronization scenarios.

17. When should you use `Parallel.For` or `Parallel.ForEach`?

Use them when you have sufficiently expensive, independent CPU-bound iterations that may benefit from execution across multiple processors.

They are usually poor choices for tiny operations, strongly ordered workflows, or operations dominated by asynchronous I/O.

18. Does `Parallel.ForEach` guarantee order?

No.

Iterations aren't guaranteed to execute or complete in source order, and correctness must not depend on a particular parallel schedule.

19. Why can parallel code be slower than sequential code?

Parallelization adds costs for:

EXAMPLE / FLOW

```
Partitioning

Scheduling

Synchronization

Coordination

Contention
```

If each iteration is inexpensive, those costs can exceed the saved CPU time.

Microsoft therefore recommends measuring rather than assuming parallel execution improves performance.

20. What is `Parallel.ForEachAsync`?

It is an asynchronous parallel iteration API whose body returns an awaitable `ValueTask`.

It can process enumerable or asynchronous-enumerable inputs while controlling concurrency through `ParallelOptions`. Default overloads execute at most processor-count operations in parallel.

21. What is PLINQ?

PLINQ is Parallel LINQ.

You normally opt into it using:

```C#
source.AsParallel()
```

It is aimed at parallelizing CPU-intensive LINQ-to-Objects processing.

It introduces partitioning and synchronization overhead, so it isn't automatically faster than sequential LINQ.

22. Can you run multiple EF Core operations in parallel on one DbContext?

No.

EF Core explicitly does not support multiple concurrent operations on one `DbContext`, including parallel async queries and concurrent use from different threads.

Await operations sequentially or deliberately use separate context instances.

23. Why is excessive Task.Run usage problematic in ASP.NET Core?

ASP.NET Core already uses thread-pool resources to process many concurrent requests.

Moving request work into `Task.Run` doesn't create extra CPU capacity and can add thread-pool contention.

Naturally asynchronous I/O should be awaited directly.

24. What is the safest general strategy for shared state?

The best strategy is often to avoid sharing mutable state.

Prefer:

EXAMPLE / FLOW

```
Immutable objects

Method-local state

Independent snapshots

Clear ownership

Message passing
```

before adding locks.

Synchronization is necessary when sharing cannot be avoided, but less sharing usually means fewer concurrency bugs.

25. How would you design concurrent processing in the Fundraising Platform?

A practical approach is:

EXAMPLE / FLOW

```
Database I/O
    →
Use async EF Core sequentially
on each DbContext

Independent CPU-heavy calculation
    →
Materialize immutable snapshots
then consider Parallel.ForEach

Shared simple counters
    →
Interlocked

Shared multi-field invariant
    →
```

```
lock

Bounded async external calls
    →
SemaphoreSlim or Parallel.ForEachAsync

Cross-server database invariant
    →
Database concurrency/transaction mechanisms,
not process-local locks
```


The correct tool depends on **what is shared, whether the work is CPU- or I/O-bound, and where the concurrency boundary exists**.

Quick Revision

Concept	Purpose
Concurrency	Overlapping work
Parallelism	Simultaneous execution
Async	Non-blocking waiting
`Thread`	Execution path
ThreadPool	Runtime-managed worker threads
`Task`	Represents operation/completion
Race condition	Timing-dependent incorrect result
Atomic operation	Indivisible synchronization operation
`Interlocked`	Atomic scalar operations
`lock`	Synchronous mutual exclusion
`System.Threading.Lock`	Modern dedicated lock object
`Monitor`	Lower-level lock/condition primitive
`SemaphoreSlim`	Limit concurrent entrants / async gate
`volatile`	Specialized memory visibility semantics
Deadlock	Cyclic waiting
Livelock	Active but no progress
Starvation	Unable to obtain resources
`Parallel.For`	Parallel indexed CPU loop
`Parallel.ForEach`	Parallel enumerable CPU loop
`Parallel.ForEachAsync`	Bounded async parallel iteration
PLINQ	Parallel LINQ-to-Objects
`MaxDegreeOfParallelism`	Bound parallelism
Thread confinement	Avoid sharing state
Critical section	Code requiring exclusive access

Complete Fundraising Example

Suppose the Fundraising application needs to generate a risk score for 25,000 Gifts and then send selected notifications.

The workflow has three very different stages:

```
EXAMPLE / FLOW
1. Read data from SQL Server

2. Perform CPU-intensive score calculation

3. Send notifications through an HTTP API
```

The concurrency strategy should differ for each stage.

Stage 1 — Database I/O

```
C#
List<GiftCalculationInput> inputs =
    await dbContext.Gifts
        .AsNoTracking()
        .Where(
            gift =>
                gift.TenantId == tenantId)
        .Select(
            gift =>
                new GiftCalculationInput(
                    gift.GiftId,
                    gift.TotalAmount,
                    gift.GiftDate))
        .ToListAsync(
            cancellationToken);
```

This stage is:

```
EXAMPLE / FLOW
I/O-bound
```

Use:

```
EXAMPLE / FLOW
async / await
```

Do not wrap it in:

```
C#
Task.Run
```

and do not start parallel queries on the same context.

Stage 2 — CPU Calculation

Suppose:

```
C#
CalculateRiskScore(...)
```

performs expensive CPU work.

Use independent results:

```C#
GiftScore[] scores = new GiftScore[inputs.Count];

ParallelOptions options =
    new()
    {
        MaxDegreeOfParallelism =
            Environment.ProcessorCount,

        CancellationToken = cancellationToken };

Parallel.For(
    0,
    inputs.Count,
    options,
    index =>
    {
        GiftCalculationInput input = inputs[index];

        scores[index] = CalculateRiskScore(input);
    });
```

Each worker writes only to its own array index.

There is no shared:

```C#
List<GiftScore>.Add(...)
```

race.

Stage 3 — External HTTP Notifications

Suppose notifications should be sent to an external service but no more than eight at once.

```C#
GiftScore[] actionable = scores.Where(score => score.RequiresNotification).ToArray();

ParallelOptions notificationOptions =
    new()
    {
        MaxDegreeOfParallelism =
            8,

        CancellationToken = cancellationToken };

await Parallel.ForEachAsync(
    actionable,
    notificationOptions,
    async (
        score,
        token) =>
    {
        await notificationClient.SendAsync(
            score.GiftId,
            token);
    });
```

This stage is:

EXAMPLE / FLOW

```
I/O-bound
+
bounded concurrency
```

THREADS / PARALLELISM

not CPU parallelism.

End-to-End Flow

EXAMPLE / FLOW

```
SQL Server
    ↓
await asynchronous query
    ↓
Immutable/local Gift inputs
    ↓
CPU-heavy calculation
    ↓
Parallel.For
    ↓
Calculated scores
    ↓
Select actionable items
    ↓
Parallel.ForEachAsync
with max concurrency = 8
    ↓
External notification service
```

Different problems require different concurrency tools.

Final Decision Flow

EXAMPLE / FLOW

```
Need operation to wait for I/O?
    ↓
async / await
```

EXAMPLE / FLOW

```
Need one simple shared counter?
    ↓
Interlocked
```

EXAMPLE / FLOW

```
Need a short synchronous
multi-step critical section?
    ↓
lock
```

EXAMPLE / FLOW

```
Need async exclusive access?
    ↓
SemaphoreSlim(1,1)
```

EXAMPLE / FLOW

```
Need at most N async operations?
    ↓
```

```
SemaphoreSlim
or Parallel.ForEachAsync
```

EXAMPLE / FLOW

```
Need CPU-heavy independent iterations?
    ↓
Parallel.For / Parallel.ForEach / PLINQ
```

EXAMPLE / FLOW

```
Need database operations
on one DbContext?
    ↓
Do not parallelize them
```

EXAMPLE / FLOW

```
Need shared concurrent collection?
    ↓
Topic 29
```

Final Concurrency Mental Model

EXAMPLE / FLOW

```
Start with:
"No shared mutable state"
        ↓
Can that design work?
    ├─ Yes
    │   ↓
    │ Prefer it
    │
    └─ No
        ↓
What is shared?
```

For one scalar:

EXAMPLE / FLOW

```
Interlocked
```

For multi-step synchronous state:

EXAMPLE / FLOW

```
lock
```

For async gating:

EXAMPLE / FLOW

```
SemaphoreSlim
```

For CPU data parallelism:

EXAMPLE / FLOW

```
Parallel / PLINQ
```

For I/O concurrency:

EXAMPLE / FLOW

```
Task.WhenAll
or bounded async concurrency
```

THREADS / PARALLELISM

Then always ask:

EXAMPLE / FLOW

```
Can this deadlock?

Can this race?

Can this starve threads?

Is ordering required?

Is state actually thread-safe?

Is the DbContext shared?

What happens under 100 concurrent requests?

Is parallel execution actually faster?
```

The central lesson is:

> **Concurrency is primarily a correctness problem before it is a performance technique. Prefer eliminating shared mutable state, use the highest-level synchronization primitive that expresses the requirement, use `Interlocked` for simple atomic state, `lock` for synchronous invariants, `SemaphoreSlim` for async coordination, and TPL/PLINQ only when the workload genuinely benefits from parallel execution. In ASP.NET Core, remember that requests are already concurrent and that uncontrolled parallelization can reduce total application throughput rather than improve it.**

Next roadmap topic: Topic 29 — Concurrent and Immutable Collections.